Q-Pathformer: Multi-state Machine Learning & LLM Training.

By Anthony Siljeg

Founder of Alnotrea Labs - Q-Pathformer Machine Learning and LLM Training. Vol.1
First published on LinkedIn, December 5, 2024

Document Identifier: QPATH-ARCH-001

Document Scope
This document describes the conceptual architecture of the Q-Pathformer multi-state machine learning model.
It presents an execution architecture for organizing training data, token clusters, and multimodal agents across multiple learning states.
Governance frameworks such as Access-PoD may govern datasets consumed by this architecture, but governance authority remains external to the execution environment described in this report.

Architecture Overview

This document describes the conceptual architecture of the Q-Pathformer multi-state machine learning model. The architecture applies quantum-inspired computational principles while operating on classical hardware. This architecture maintains a similar scale and structural design to the model described below. The concept primarily focuses on quantum computing in machine learning and LLM training with the potential for hybrid approaches to offload specific parts of the computation to quantum computing. Some modifications are needed between the two computational operations.

This model involves software applications and developments for quantum computing and hardware processors based on qubits. Each of the 18 clusters contains six spikes, resulting in 108 spikes of training data. Additionally, each of the six multimodal agents has six spikes, contributing another 36 spikes of training data, a total of 144. Each data point spike represents a model for quantum computations by allocating 144 qubits. The entanglements on the surface integrations are based on a multi-state Q-Pathformer. Furthermore, unutilised spiked data can be further represented for further computation, such as simulations or data weights consensus approaches. Scaling principles of this architecture would require enlarging the number of token clusters and processors with more cubits while trying to achieve the same coherence or

improvement with scaling. Current LLMs based on classical computation far surpass quantum computing for training LLM simply because they can process billions of parameters and tokens to train and fine-tune LLMs.

Quantum systems need a scalable quantum random-access memory (qRAM) system and effective backpropagation techniques, among others, to handle the demands of billions of parameters and memory for training and fine-tuning LLM effectively.

This system uses the foundations of a previous model I open-sourced two years ago (vol.1 Access PoD) and has kept the original design. I've included references below.

Abstract

We propose a neural network architecture Q-pathformer sphere representation for machine learning and training LLMs with three specific states. Within the **first primary State 1**, the fixed 3D sphere - 18 rotating token clusters of data consist of flat data in the form of a 2D regular hexagon and a complex hexagon of spiked data for machine learning and training Large Language Models.

Training vectors use two primary embedding states with a **3D Cube (state 2)** and an inner **3D Q-sphere (state 3)** of synthetic data. The superpositions on the Q-sphere pre-training matrices are quick labels of multi-head vector weights and Cube is for grouping vector weights input training (cubit coherences) with superpositions towards the surface of the fixed sphere State 1. The embedding of spiked data is for training and finetuning data token clusters State 1. Token clusters (Data) merge copies of synthetic datasets training with eight priority classes (State 3) based on input pre-training. Class 1 data (state 3) is the gold standard for further model training, and class 8 is the least influential class for new inputs and pre-training (state 3). The Q-path-former-queue sequential flow dynamics 3,2,1 & 1,2,3 states of machine learning, **State 3** pre-training - **State 2** trained data - **State 1** finetuning data (spiked data and computations) integrations with pre-trained Multimodal agents for entanglements of superpositions. Sequence flow **State 1** involves agents, token clusters, and spiked data. In **State 2**, the focus shifts to optimizing token range and usage by balancing consensus with the importance of data. This includes merging synthetic datasets to facilitate training in state 3. **State 3** develops qRAM and synthetic datasets organized into eight data classes for context retention, enhancing understanding of variables during pre-training tasks with new inputs.

Introduction

AI systems have numerous components and architectures. This concept outlines a unified model for machine learning and LLM training systems with pre-trained multimodal agents.

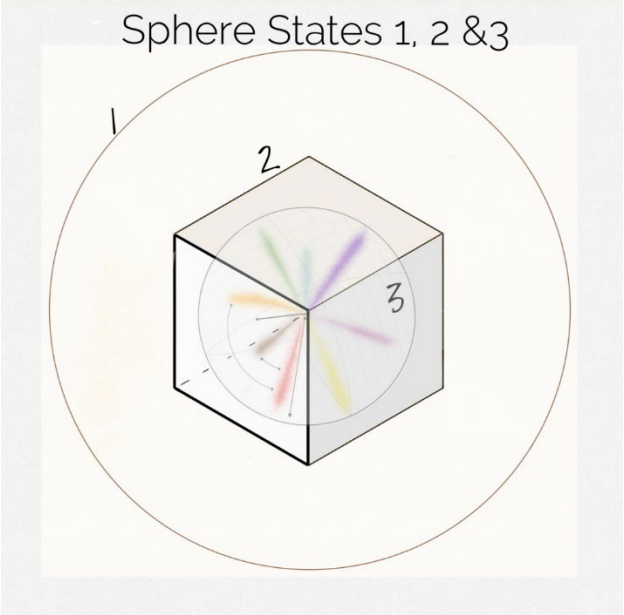


Image created by the author

- One attention mode (primary state 1) - 3D Sphere comprises multimodal agents and several token clusters (memory storage) with fine-tuning for model updates.
- One attention mode (state 2) - inner Cube vector training of grouping weights for fine-tuning with superpositions.
- One attention mode (state 3) - the inner Q-sphere vector - pre-training and quick label of weights with superpositions.

1. Memory and Computational Operating expense

The complexity of hierarchical data structures often requires more memory with common computational resources. For instance, representing and traversing a tree requires more memory than flat data, and the computation to train models can become much more expensive.

In our conclusion, hierarchical data structures introduce complexity due to their layered nature, the dependencies between levels, and the difficulty in processing such data effectively. Specialised models and algorithms are often required to handle these challenges, adding further complexity to the machine-learning pipeline.

- **1 of 18 data cluster categories – 2D regular hexagon (shape) state 1.**

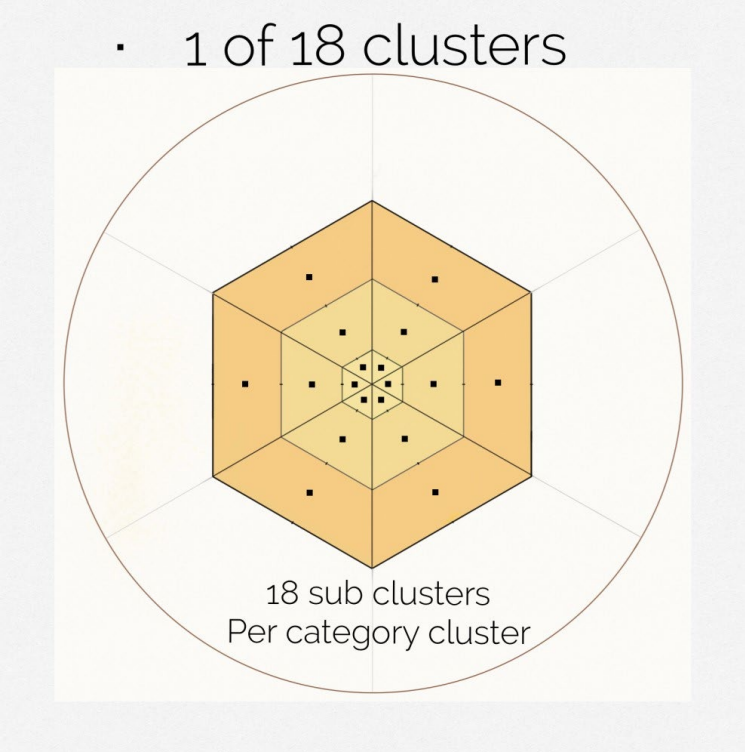


Image created by the author

- **One cluster category and spiked data - 2D Complex hexagon (shape) state 1.**



Image created by the author

Some examples of 18 Major categories commonly used for pretraining large language models (LLMs):

- General Knowledge: Wikipedia, encyclopedias, and other factual resources.
- News Articles: News outlets, magazines, and journalism websites.
- Books and Literature: Fiction, non-fiction, classical, and modern literature.
- Scientific Papers: Research articles and academic papers.
- Technical Documentation: Manuals, software documentation, and coding examples (e.g., GitHub).
- Web Content: Blog posts, forums, and general web pages.
- Social Media: Posts and discussions from Twitter and Reddit platforms.
- Legal Documents: Case law, statutes, contracts, and other legal texts.

- Medical Texts: Medical textbooks, case reports, and healthcare guidelines.
- Financial Reports: Business reports, financial statements, and market analyses.
- Code and Programming Languages: Source code from various languages and frameworks.
- Dialogues and Conversations
- Product Reviews: Consumer product reviews, feedback, and ratings.
- Historical Texts
- Cultural Texts
- Philosophy and Ethics
- Policies, government publications, and public records.
- Educational Materials

2. Multimodal Agents.

Six multimodal agents are used to pre-train the LLM system and provide the varied training data necessary to develop comprehensive large language models (LLMs) that effectively manage different inputs and outputs. Each of the six agent modes additionally utilises a computer vision pathformer algorithm*. (image recognition)—intra-class variations/separations, feature extraction, pose differences, illumination and colour transitions.*
Q-pathformer: Unit—Group—Colour—Code—Measure—Train (UGCCMT).

All units within states 1, 2, and 3 have different units, colours, codes, measures, train-tag **identifiers, and training properties.**
All operations of the 6-agent model involve merging (supervising) agents and clusters through the Q-Pathformer gateway. (Sphere State 1). Model training and data integrations involve pre-trained multimodal agents and prototyping mode (updates), facilitating system machine learning through pre-training and fine-tuning data. The Q-pathformer gateway manages memory hierarchy and spiked memory outputs and confirms the integration of new data weights (described in 3.1).

Further multimodal agent integrations may follow the same processing and curation procedures described in the sections below.

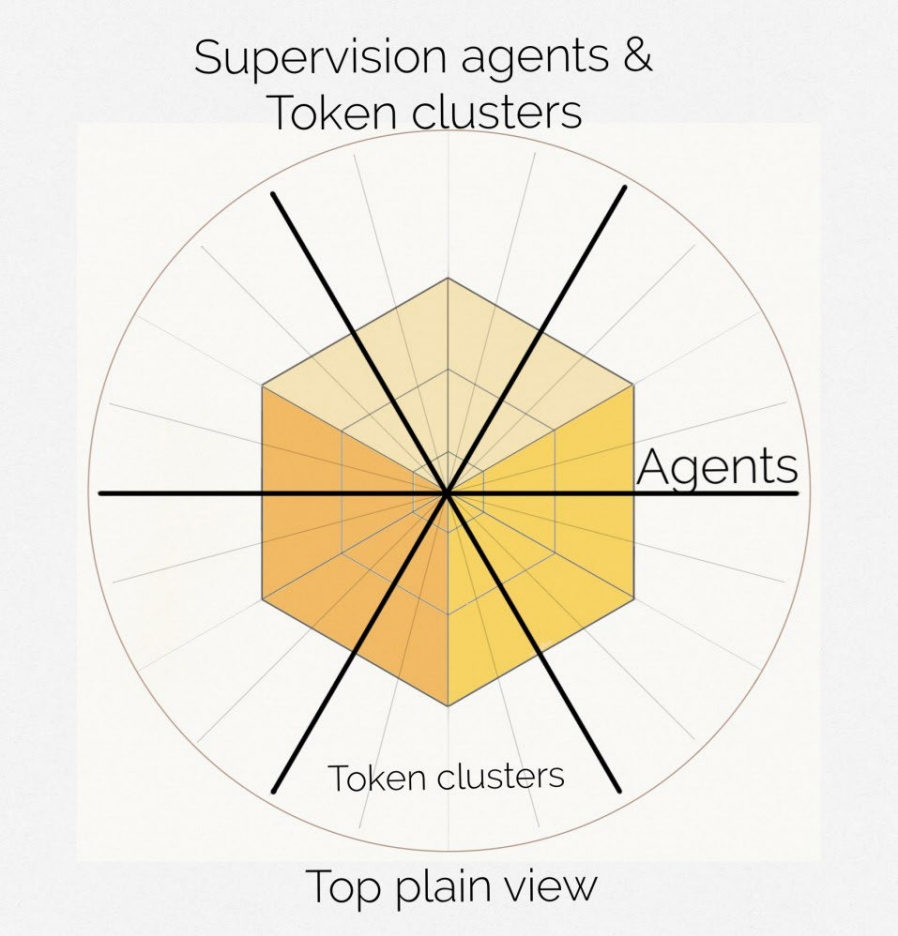


Image created by the author

1. **Text-Image Agents**: These agents link text descriptions with corresponding images, helping the model understand the visual context and generate descriptions.
2. **Text-Audio Agents**: Agents that associate spoken language (audio) with transcriptions (text), enabling models to process both written and spoken communication.
3. **Text-Video Agents**: These agents map video content to text, enabling LLMs to generate captions or interpret visual scenes in video form.
4. **Text-Speech Synthesis Agents**: Agents that convert written text into synthetic speech, helping models learn to generate realistic audio from text.
5. **Text-Action Agents**: In interactive environments, link textual instructions with actions performed in virtual spaces or games.
6. **Text-Sensor Data Agents**: Agents that link textual data with sensor outputs (like temperature, pressure, or motion), helping LLMs learn to interpret real-world sensor data alongside descriptive text.

2D regular hexagon – Q-pathformer data trains - Unit-group-colour-code-measure-train (UGCCMT) training loops.

- 3 Pathformer loops

- 1 of 6 agent assistance unit-groups
- 6 Agent Duo measures unit-groups

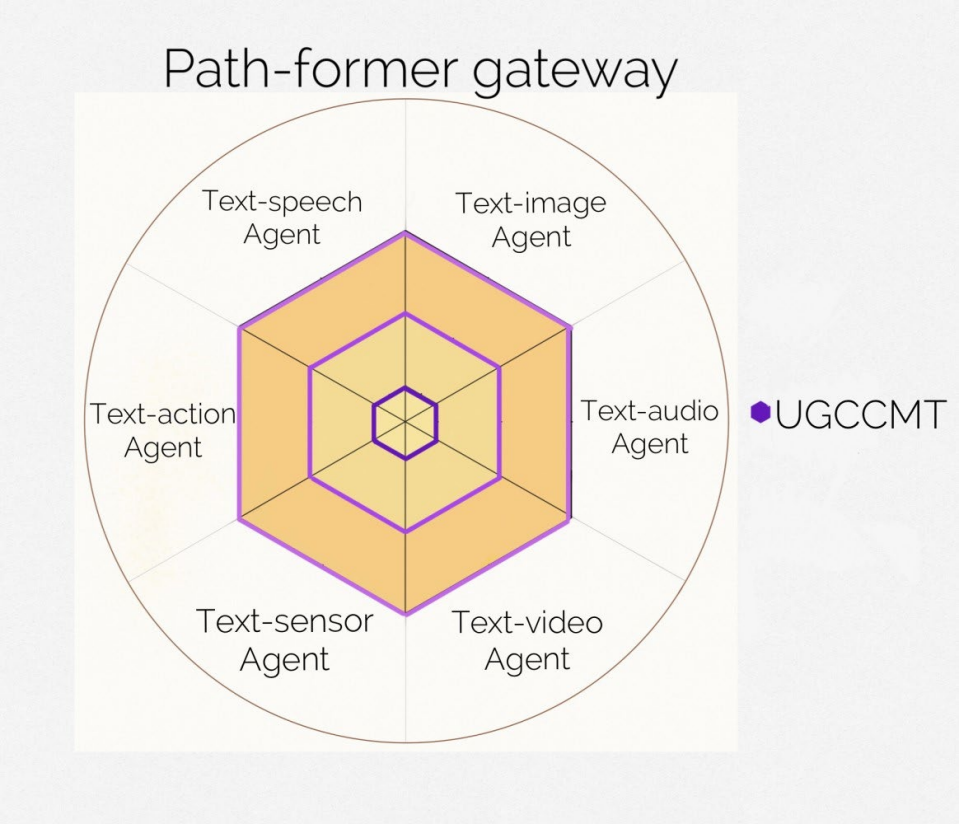


Image created by the author

Input – Training – Fine-tuning Output data. (Supervised and unsupervised machine learning in section 5. below).

- Input - Collection of data inputs and identification of data weights for data type allocations. (Inner loop units)
- Training – data sampling, metadata and token selection. (Middle loop units)
- Output – Token cluster categories processing data for further finetuning and weights of unit-group and token groups.

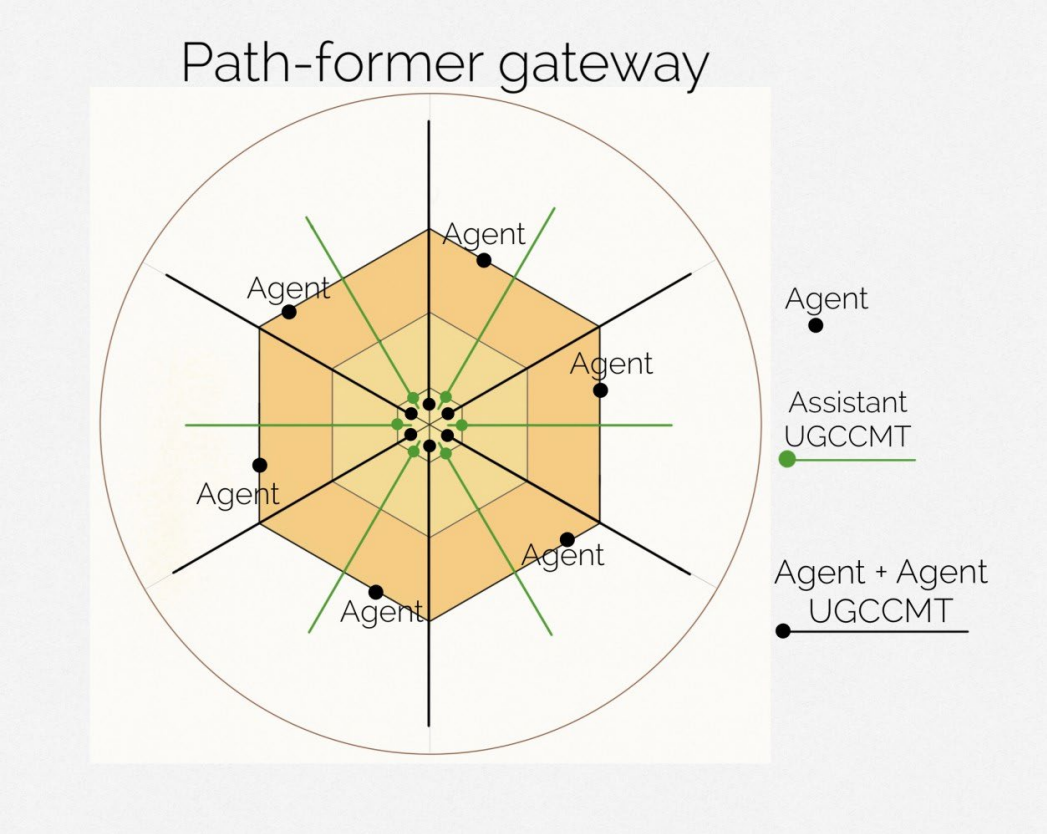


Image created by the author

- Agent unit-group assistants. (Green UGCCMT line)
- Units and groups Agent **duo** measure (Black UGCCMT line)

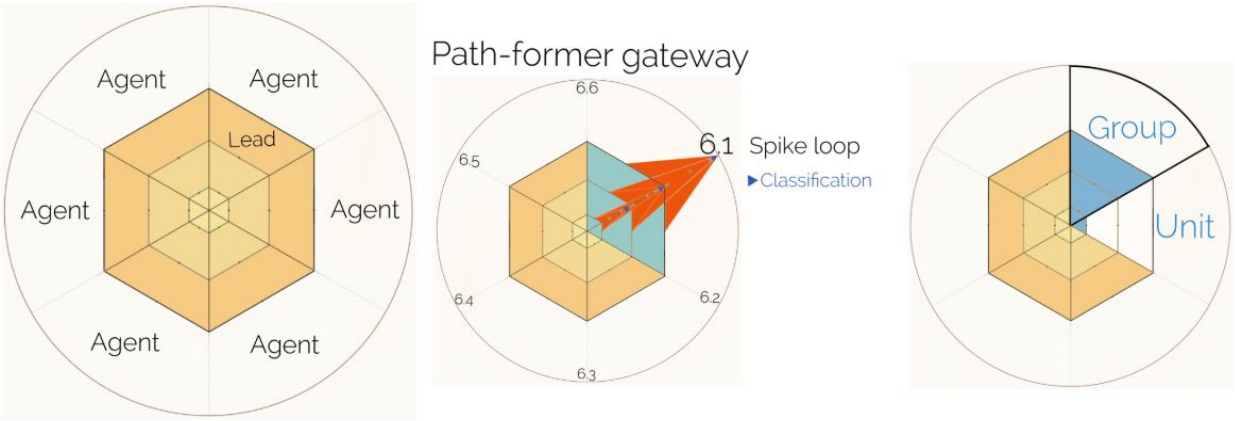


Image created by the author

3.0. Flat data memory hierarchy (State 1)

One token cluster category (location) of spiked data (complex hexagon past the regular hexagon), and qRAM on the outer sphere from states 3 and 2.

Different stages of data-spike - backpropagation, analysis, and rotation stream monitoring.

- **New unit data point—**spiked data (direction towards the surface). Agent attention analysing Data. The unit token is sent to the outer sphere (State 1) as working memory and new data processing for updating data weights allocation to a selected token and cluster.
- **Confirmed data -** Spiked data on (state 1) from superpositions (state 3 and 2) to the outer sphere for processing. (more details in section 4.1 below)
- **Data UGCCMT** (hexagon) 6 data collection points (direction inwards towards the cluster) and allocation for cluster category State 1.

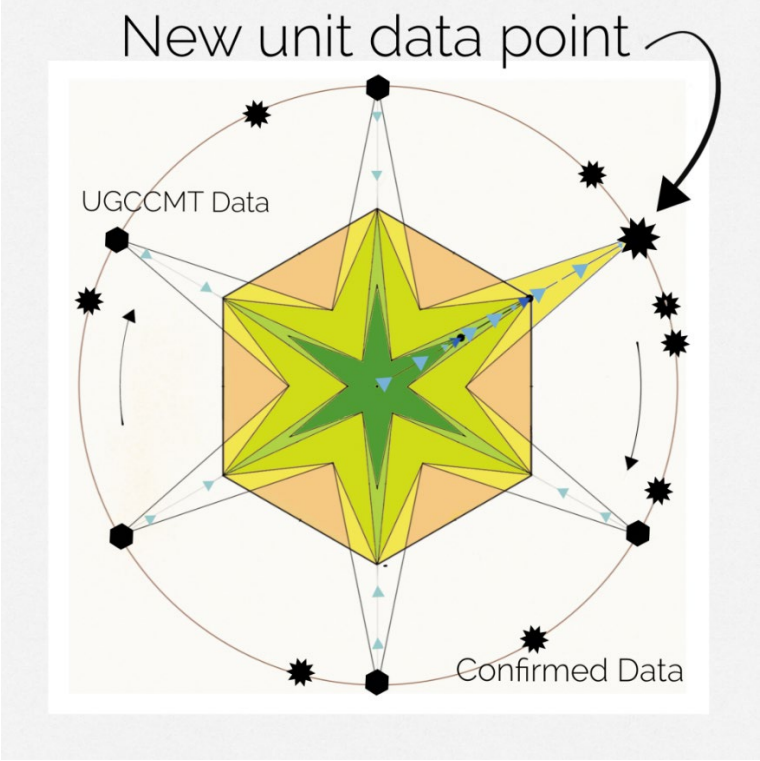


Image created by the author

The Q-pathformer States process spiked data using a three-phase training approach that incorporates datasets and qRAM backpropagation.- states 3 and 2.

- In the first phase, input pre-training and spiked data from 72 superpositions (single cubit) and **unit data entry** (below chart) backpropagation for,

- In the second phase, trained spiked data from 24 superpositions (double cubit), for **unit data entry** (below chart) backpropagation for,
- The third phase, training and spiked data from 8 superpositions (triple cubit) DATA training backpropagation for,
- **Unit data entry** state 1 – backpropagation and finetuning involves cluster rotations (all phases above) and spiked data matching UGCCMT. This includes fine-tuning and processes from input query docs and SQL query loop tracing UGCCMT-specific database token-sub-token selection and allocation (state 1). ***(Supervised and unsupervised machine learning in section 5. below).***
- Finetuned Token clusters (Data) merge copies of synthetic datasets training with eight priority classes for (State 3) based on input training phase 1.

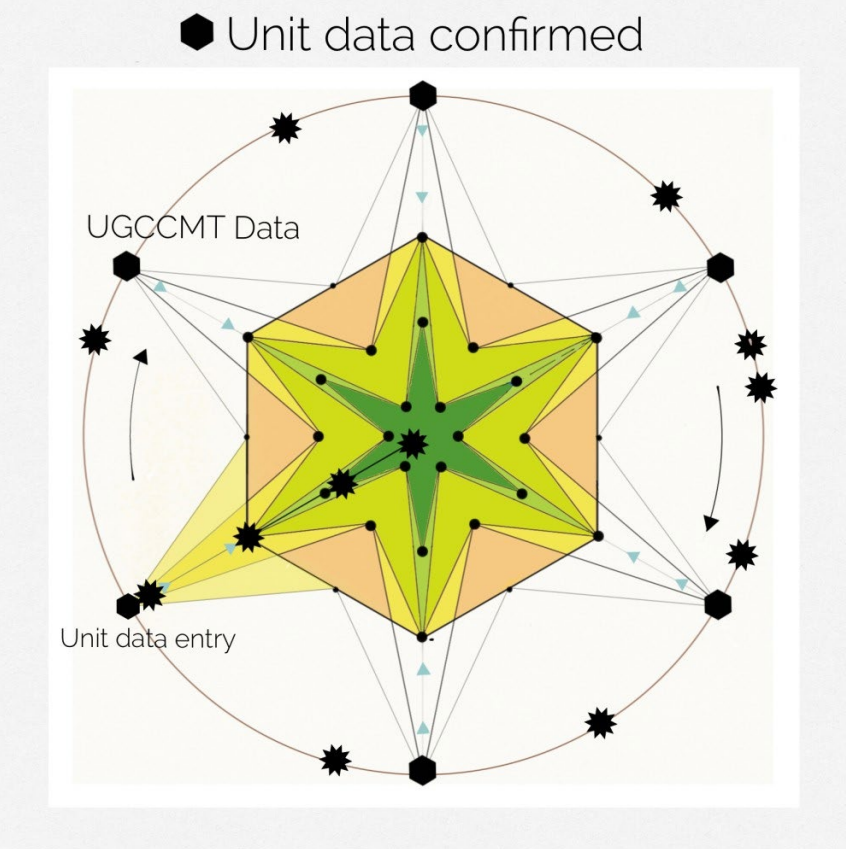


Image created by the author

3.1 Seeking Consensus – Confirming New Data Weights

The process of seeking consensus refers to users and multimodal agents working collaboratively to confirm whether the new data should be integrated into the model. Specifically, the system decides how much influence (weight) new or updated data should have on the existing model.

The multi-head vector weights control how much impact this new information will have on future predictions or decisions made by the model.

Data Weights (numerical values) determine the importance of specific data in shaping the model's behaviour. Fine-tuning involves adjusting the weights of the new data.

- Users: Human feedback labelling of data to help guide the fine-tuning process.
- Multimodal Agents System to process different types of inputs (text, images, audio, etc.) and help understand and validate the new data together to assess the new unit data from multiple perspectives or modalities (e.g., text-to-image understanding, voice-to-text conversion, etc.)

Token Allocation to Token Cluster Categories
In natural language processing (NLP), models break down text into smaller tokens. Tokens are then grouped into token clusters, representing categories or information themes (e.g., finance, policy, health, general conversation).

When fine-tuning with new data, the system tries to determine how the new tokens should be categorized into existing clusters or whether they relate to other sub-clusters. This ensures the model understands how to classify new or unconfirmed UGCCMT data/information or metadata based on context and relevance.

Example of Process Flow
1. New Data Entry: A user provides instructions, and the agent trains on new data, such as new sentences and image-text pairs, for training purposes.

2. Fine-tuning: The system adjusts its model parameters and data weights to incorporate the new information.

3. Multimodal Consensus: Users and multimodal agents review the new data, ensuring it fits into the correct categories and doesn't disrupt existing patterns.

4. Token Allocation: The system assigns the new tokens to the appropriate token cluster categories and sub-clusters (e.g., grouping tokens related to finance with other financial terms with a UGCCMT. ***(Supervised and unsupervised machine learning in section 5. below).***
5. Confirmation: Once a consensus is reached, human feedback loops on existing data can confirm new data trained can be fully integrated into the model, and the system updates its

understanding accordingly for further training. This process allows the model to stay up-to-date, dynamically adjusting to new information while maintaining accuracy across various token cluster categories.

4.0 Q-Pathformer sphere coordinates and positions.

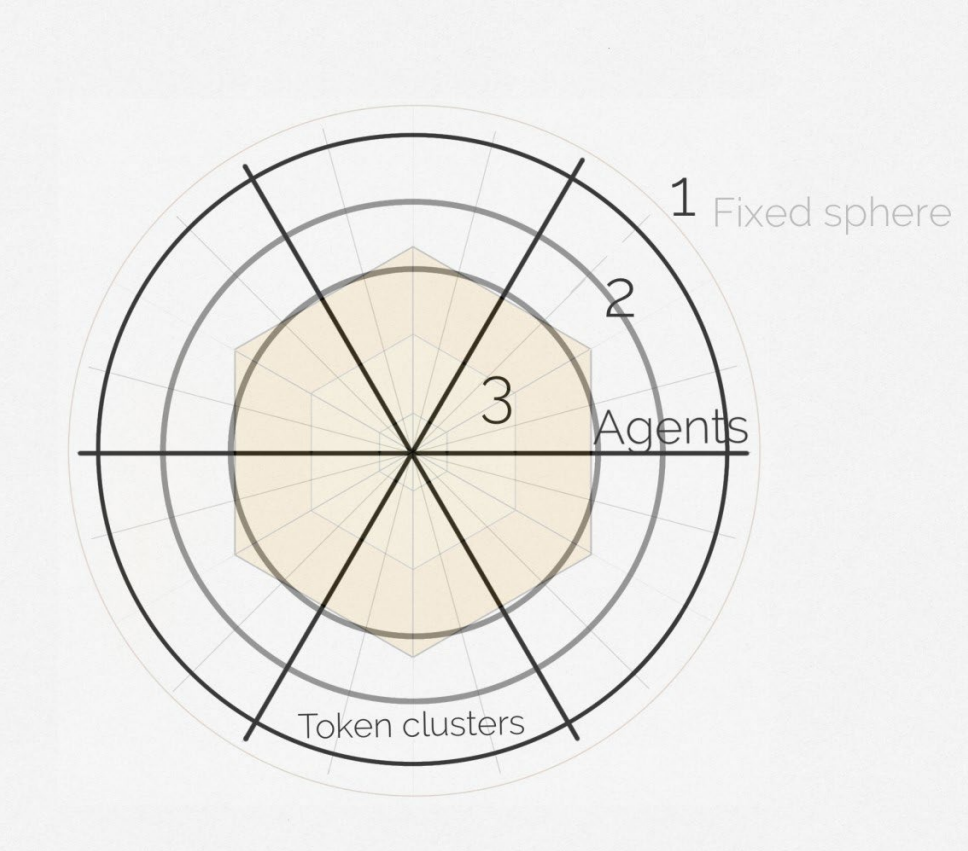


Image created by the author

1. The 3D sphere (fixed position 1 and coordinates) represents the token cluster space with multimodal agents. Inside this sphere position 2 (a central vector rotation is coordinated with **two key regions:**
2. The inner sphere (outer Layer 2) rotations (above) correlate – unit-group cluster and spike/s coordinates with the outer surface (fixed sphere state 1) coordinates for data point connections and allocations of spiked data. This is the sphere's outer surface, where token clusters are positioned. Each token cluster (a group of similar token data occupies a point or small region on the surface. The outer cluster/s coordinates represent how tokens are categorised based on similarity or relatedness to the sphere's (fixed) outer coordinates for training and fine-tuning data.
3. Inner Layer Coordinates position 3 - Moving deeper into the sphere, inner coordinates represent more refined or specific cluster token vocabularies. These inner regions further break down the broad categories into smaller, more specialised groups corresponding to subtopics or contextual variations.

All coordinates have distinct UGCCMT units, colours, codes, measures, train-tag **identifiers, and training properties (mentioned in section 2).**

4.1. Multi-Head Vector Cube & Q-sphere - states 2 and 3.

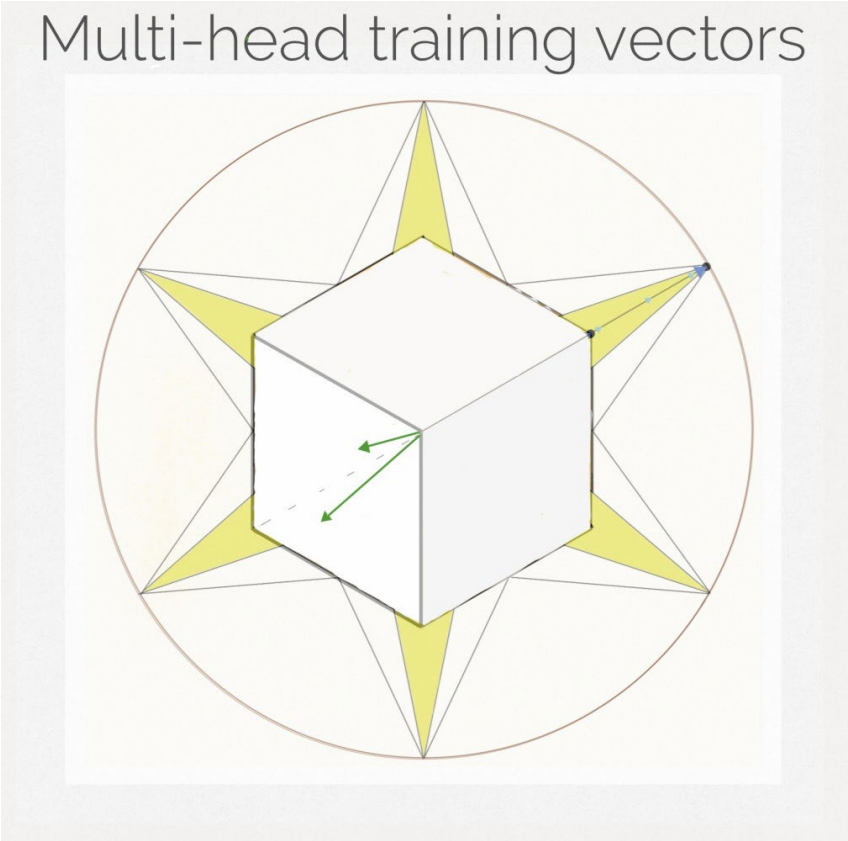


Image created by the author

- Inside the sphere's centre, there's a cube, a multi-head vector. This represents the data weights, connections between token clusters, and complex relationships between different categories.
- The multi-head aspect refers to different "attention heads", which simultaneously focus on various parts of the data. Each plane of the cube deals with specific measures of weights that adjust the data based on relationships between vectors and among different token categories.
- The internal cube interacts with the inner and outer fixed sphere coordinates, influencing how tokens are weighted and adjusted during pretraining for fine-tuning new data.
- The inner 3D cube **State 2** is different from 2D regular hexagon and complex hexagon - 1 of 18 token cluster category **State 1**. All data tokens outside the regular hexagon are

spiked data, e.g., based on input and training phase: qRAM - (quantum Random-access memory) or trained dataset spikes, synthetic datasets, for **training and fine-tuning.**

4.2 Q-sphere (State 3) measures multiple data weight measurements. For example:
The inner sphere (state 3) weight vectors are primarily used for quick labels of weights **with 72 superpositions.**

- 72 Superpostions with single-cubit gate matrices. 72 cubits are used for training small datasets (token vocabulary). **State 3**

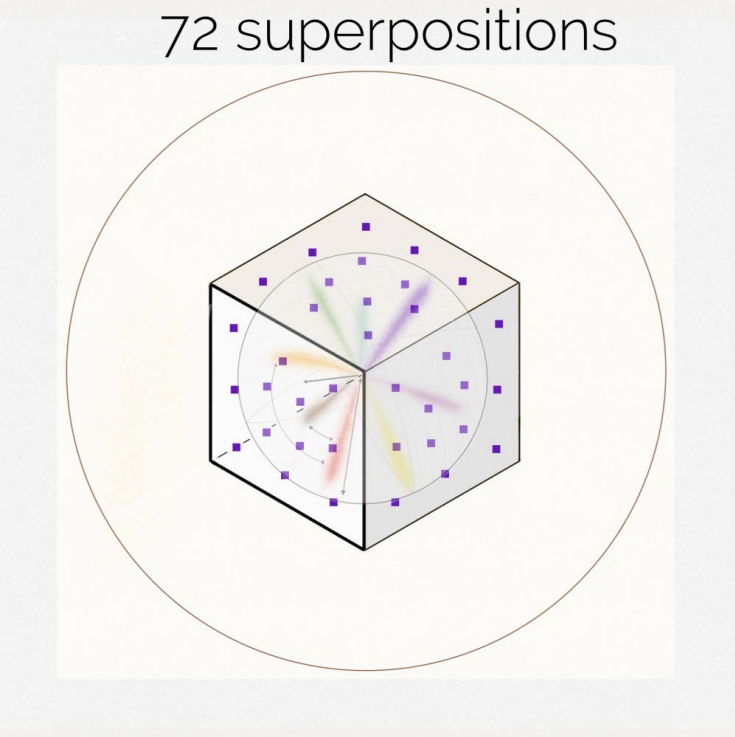


Image created by the author

Each surface (plane) of a cube form 12 superpositions.

- 24 Superpostions with double-cubit gate matrices. 48 cubits are used for training medium size datasets (token vocabulary). **State 2**

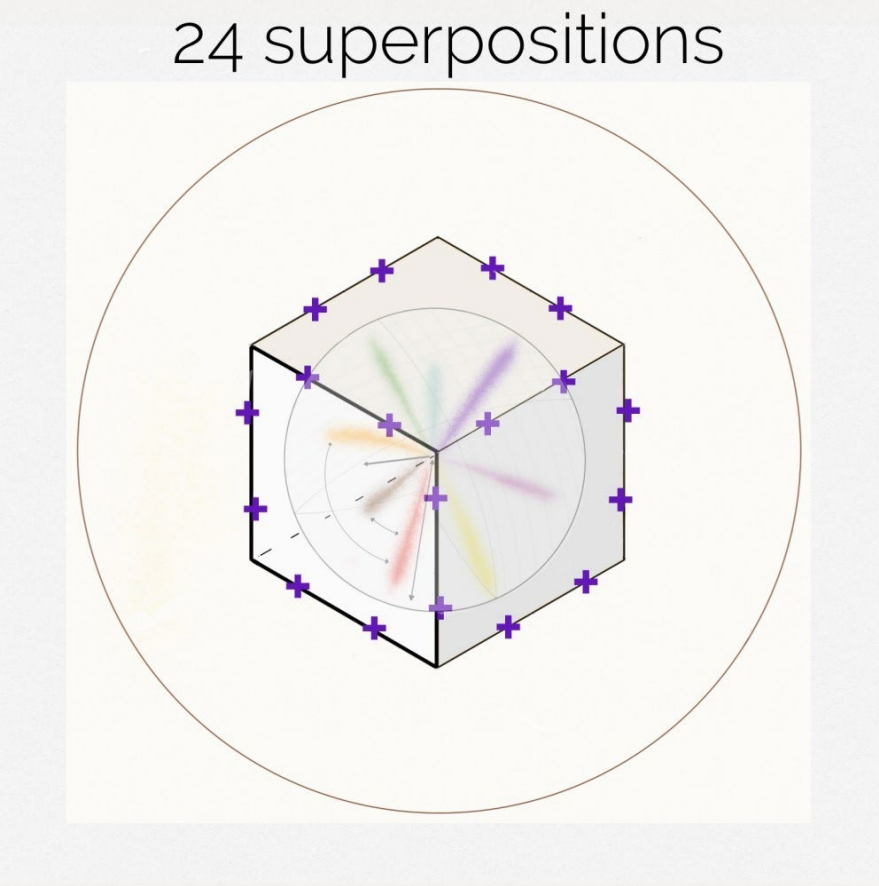


Image created by the author

Each cube's two surfaces (planes) form 24 possible superpositions.

- 8 Superpostions with triple cubit gate matrices. 24 cubits are used for large dataset (tokens vocabulary) training. **State 2**

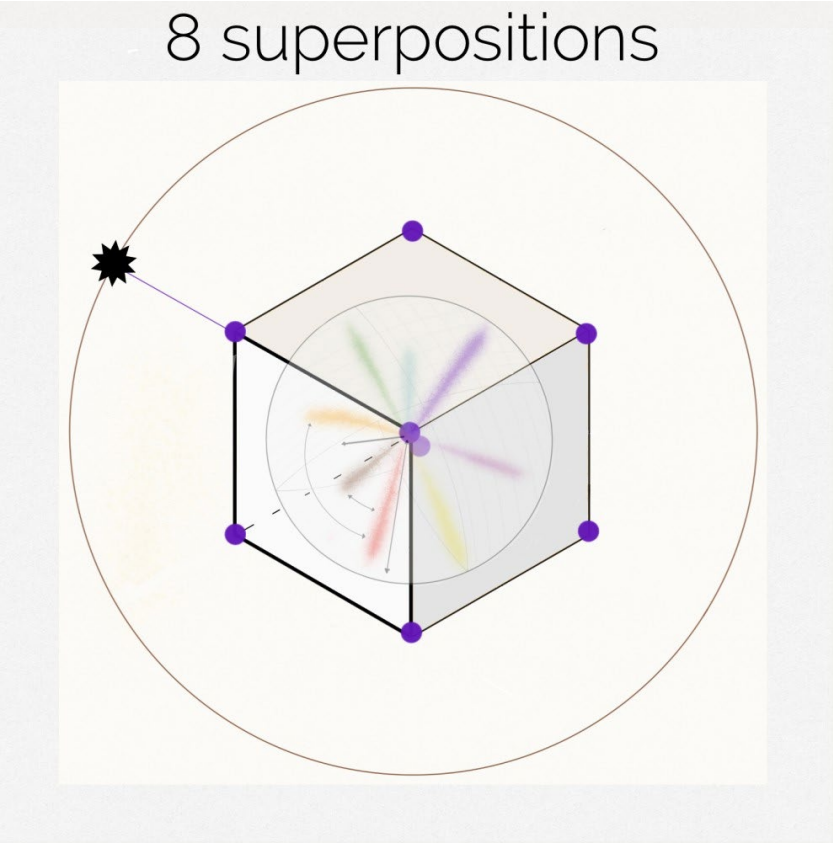


Image created by the author

The cube superpositions **(define different vectors and token data for grouping a token vocabulary).** For example, weight classes on **one or more** surfaces (planes) of a cube form 8 superpositions located from the origin of the centre 0,0 of the Cube/Q-sphere state towards each corner of a cube (8 corners) where three lines/planes of a cube meet, form 1 of 8 superpositions. A superposition is **a classed group of token vocabulary, for example, 1, 10, or 20k.**

4.3 The multi-head vectors State 2 and 3 measurement objectives and operations.

A digital gravitational force on an object with different interactions that change the motion of vectors. This represents the domain knowledge the model may learn to associate data contexts, structures, expressions, etc. **The measure of weights is** from origin 0,0, distance, weight, and distance between multi-head vectors **(number of vectors pointing towards a plane/s for weight grouping to establish superpositions of spiked data towards the outer sphere surface.**

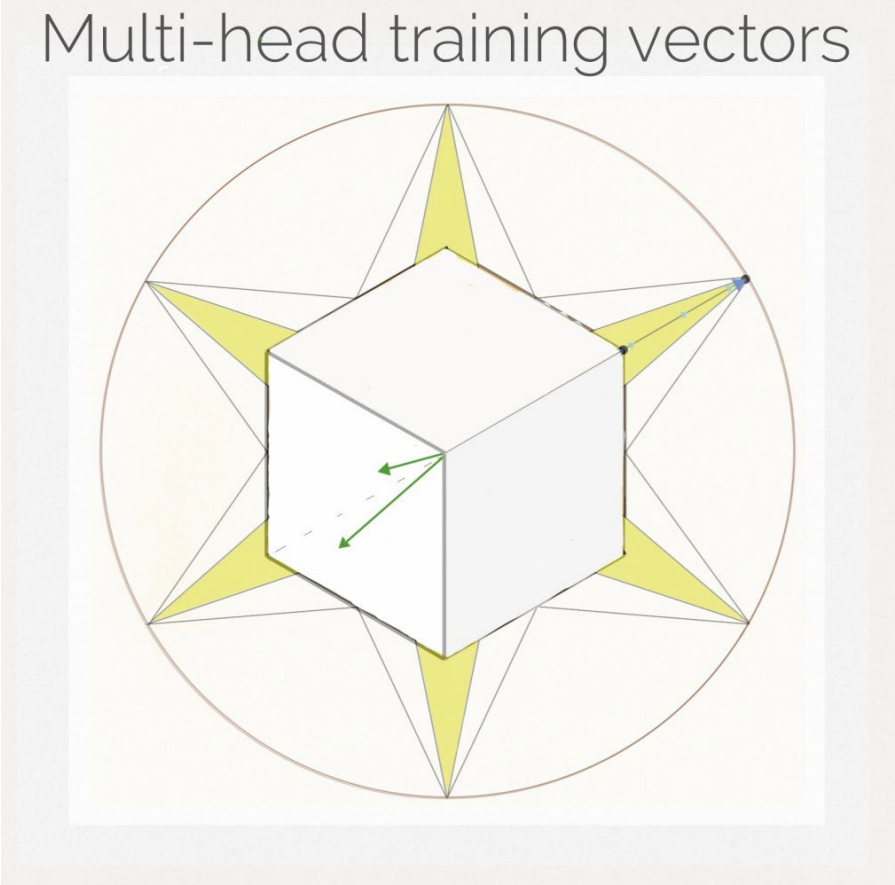


Image created by the author

The sequence queue 3,2,1 and 1,2,3 States allow the model to unify and balance broad token categories (UGCCMT flat data token clusters) and manage data weight adjustments and training via the multi-head vector cube and sphere states. For example:

- 8 superpositions (triple cubits) lead to the highest data consensus for **fine-tuning the system and data clusters** from the previously trained 24 superposition (double cubit) gates and the prior 72 superposition (single cubit) training.

Two versions of DATA (locations) for pre-training and training data.

- *Q-sphere state 3 - Use eight priority classes of synthetic data flows with 72, 48 and 8 superpositions starting from class 8 to lead class 1 gold standard data for the **pre-training and training of data**. (Described below in 4.4*
- *Q-sphere state 3—Use eight priority classes of synthetic data flows with 72 superpositions (only) starting from class 8 to lead class 1 gold standard data for data **pretraining**. Cube State 2 uses curated token cluster data (State 1) for grouping tokens and token clusters for **data training.***

4.4. State 3 is based on Qubits 0 and 1 quantum computing principles. Similarly, the Q-path former gateway interacts with states 3 and 2 for pretraining and state one fine-tuning data.
In quantum computing, *superposition* is a fundamental principle that allows quantum bits (qubits) to exist in multiple states simultaneously. Unlike classical bits, which can only represent a single state at a time—either 0 or 1—a qubit in superposition can represent both 0 and 1 simultaneously, with specific probabilities.
When a qubit is in superposition, it possesses a probability amplitude for each possible state. Quantum gates can manipulate this superposition to perform computations.

Superposition enables quantum computers to process many possible outcomes in parallel, giving them the potential to solve specific problems much faster than classical computers. This is particularly useful in **cryptography, optimization, and complex simulations.** However, Superposition only holds until a measurement is made on the qubit. At this point, it "collapses" to a definite state, either 0 or 1, providing a specific outcome for the computation.
Multimodal agents assist in processing data and entangled superpositions to transmit spiked data to the sphere.
Q-Sphere **(State 3)** North point superposition 0 and South point superposition 1 hemispheres have different conditions within (state 3) that unify with training classes of data and process for pretraining data weights.

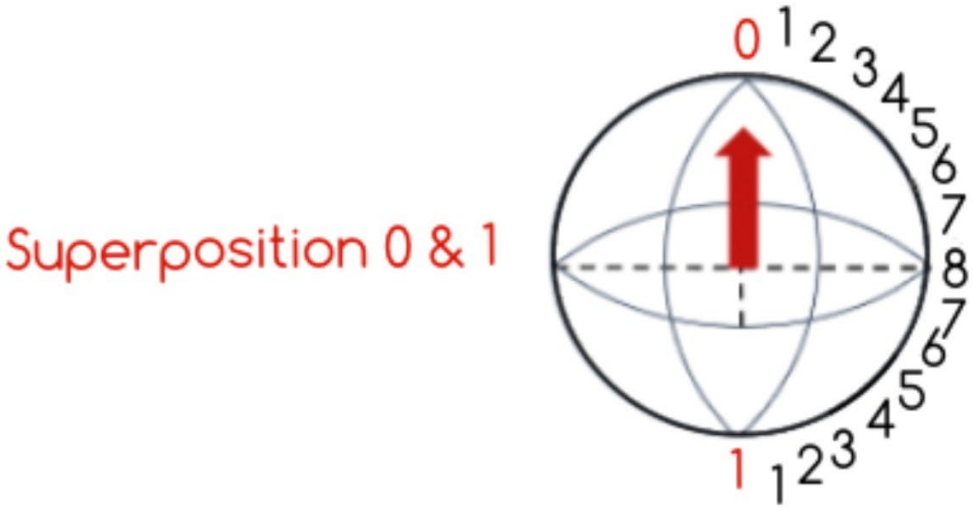


Image created by the author

- Eight major latitude data classes and eight longitude gate classifications for training weights of superpositions 0 and 1.

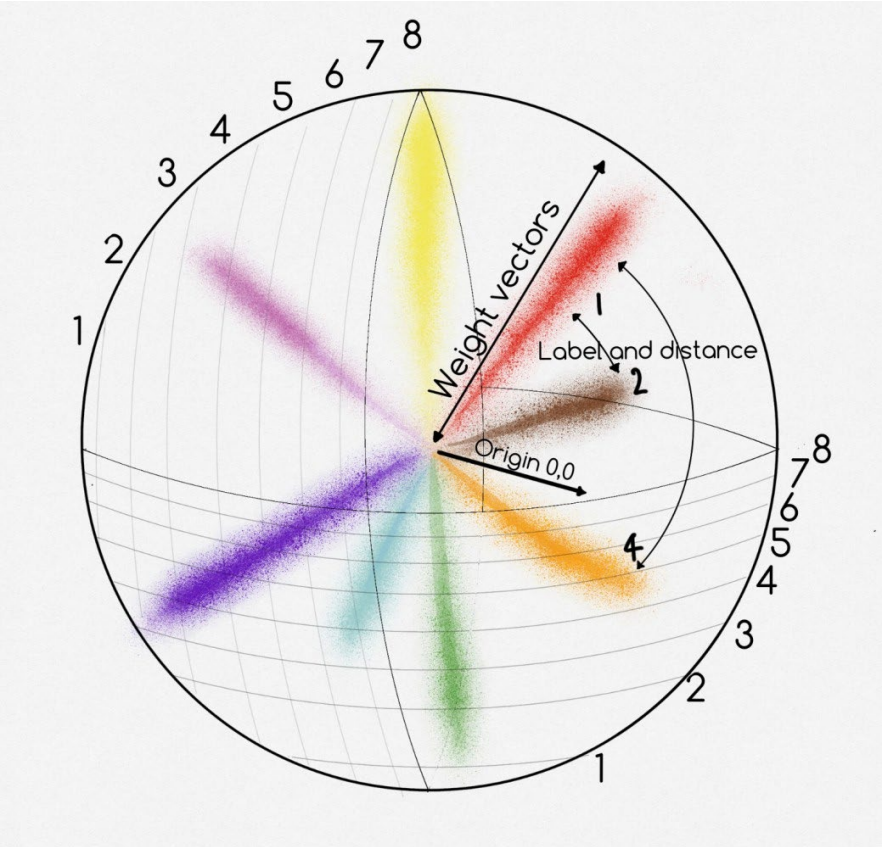


Image created by the author

All latitude and longitude coordinates on the sphere (state 3) pre-training data are different representations and behaviours. For example, class 8 at the equator is a duo latitude for both north and south classes with broad generalization of data and metadata.

- Seven North, one Equator (class 8), and seven South latitude positions form 15 latitude positions of structured classification of training data weights. Latitude positions start with the lowest priority (Class 8) located at the equator (most expansive loop) and (class 1) (smallest loop) towards North/South 0-1.
- Class 1: The smallest loop near North and South has different class hierarchies, ensuring the training process prioritizes high-quality, task-relevant data while judiciously using border or less relevant data classes to support the model's generalization. Classed data are benchmarks for processing further machine learning.

Class 8 is the primary point for new input data for pre-training (state 3) and the process for fine-tuning (state 1). Classes 7 to 1, North, and 7 to 1, South - Match classes with a polarization objective, such as Contrastive Learning. Classes are identified as similar or different, while sentiment and normality assessments are based on consensus models and approaches in each

hemisphere. Superpositions 0 and 1 outputs collapse (Data flow categories 8 classes) towards Class 1 are expressed as curated gold-standard data with trained token vocabularies/weights for Processing in Cube State 2.

Q-Sphere (State 3) 8 prominent Latitude positions of data classes.

- Class 1 (Highest Priority): This class has the most significant weight due to its importance or prevalence in the training data. The smallest loop around the sphere.

Description: Data of the utmost importance for achieving the desired outcomes, such as high-quality, highly unbiased, and precisely annotated examples.

Example: Benchmark datasets, ground truth labels, carefully curated gold-standard data.

Weight: Highest, as it directly influences the model's core capabilities.

- Class 2: Slightly less weight than Class 8 but still carries high importance.
- Class 3: Medium-high importance in the dataset.
- Class 4: Represents classes with average weight or prevalence.
- Class 5: Slightly less important than the average class.
- Class 6: Low-priority class with minimal influence on training outcomes.
- Class 7: Near the lowest priority for consideration in training.
- Class 8: Least influential class; commonly known as synthetic data and used for new input training.

Description: Unrelated or noisy data that only carry a defined training objective with input and training.

Example: Synthetic data supplementations with random online data or real-world data inputs.

Weight: Minimal (close to 0.0).

Each data class and training gate classification has different matrices for moving 1 of 8 data classes to more defined data sets for upscale/downscale data during pre-training for superpositions 0 and 1 and further training data for states 2 and 1.

Q-Sphere (State 3) 8 Longitude classifications of quantum computing gates.
From the least to the most influential gate for LLM training, it involves assessing eight classes of data of latitude positions, their importance, frequency of use, and impact on quantum computation. For example, gate 8 (longitude) in superposition 0/1 moves data in class 1 for state 2 cube (plains). Classification gates 1 on each side of the sphere (longitude) move data in class 8 to the equator.

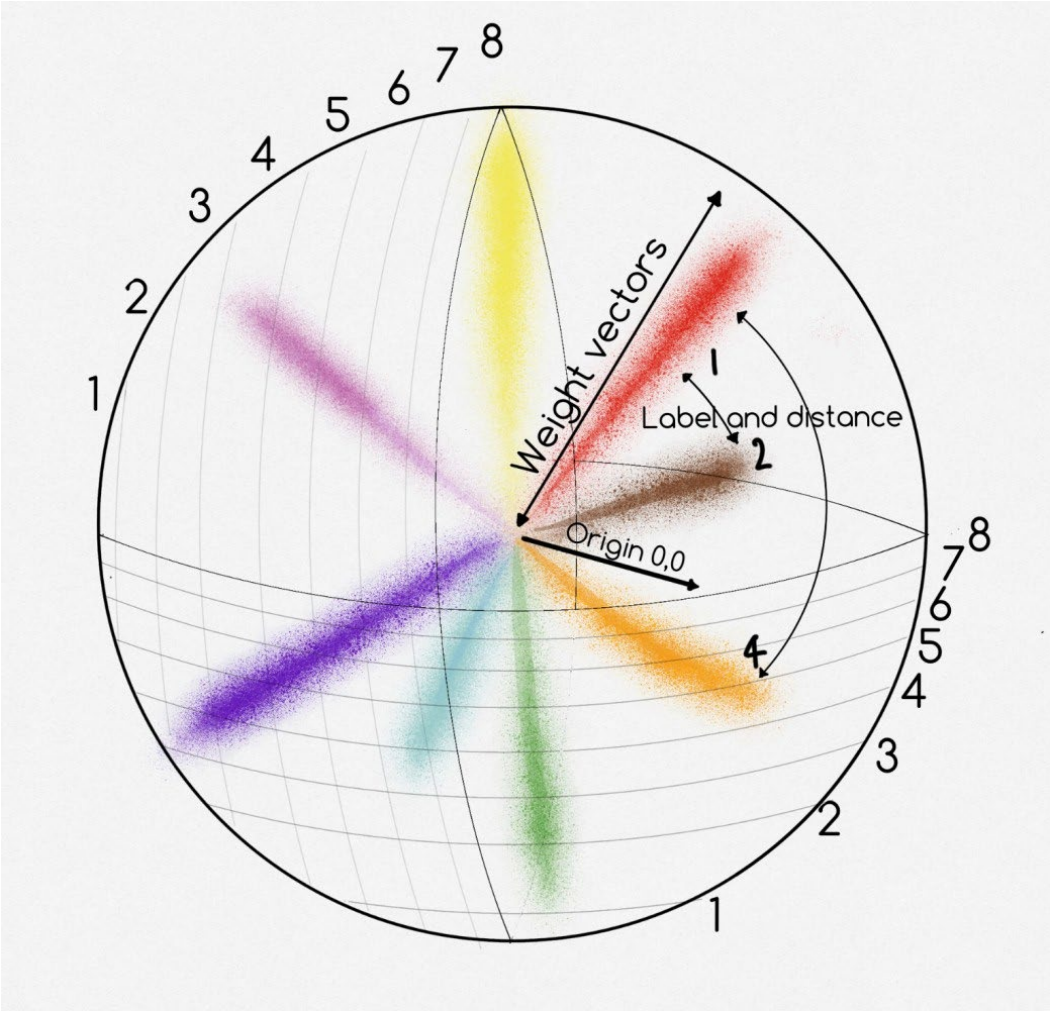


Image created by the author

Classification gate 1 - Measurement and Reset Gates (Least Influential)

- **Description**: Allow reading the state of a qubit or resetting it to a known state.
- **Influence**: Essential for interpreting computation results but does not directly affect quantum operations during runtime.
- **Examples**: Measurement gates on a computational basis.
- **References**: Widely used in post-processing but less involved in algorithmic complexity

8. Single-Qubit Gates (Most Influential)

- **Description**: Basic operations on single qubits, pivotal for state manipulation and algorithm construction.
- **Influence**: Most versatile and frequently used gates, foundational for all quantum computations.
- **Examples**: Hadamard (H), Pauli (X, Y, Z).
- **References**: Integral to creating superposition, interference, and quantum transformations

I've included references below for gate classifications.

5. Supervised and unsupervised machine learning of spiked data embedding. State 1

Agents and assistants data sampling, labelling, token categories, and clusters for data embedding.

Supervision 6 Agent Cluster - a computer vision pathformer algorithm*. (image recognition of clusters)—intra-class variations/separations, feature extraction, pose differences, illumination and colour transitions.*
Q-pathformer: Unit—Group—Colour—Code—Measure—Train (UGCCMT) tags identifiers and training properties of spiked data embedding.

Modes:

- **6 Agents**
- **One agent with 5 assistants**

Path-former gateway data sorting assistants/agents management: 3-2-1, 3-3-1, 4-1-1, 4-2-1, 5-1-1

Examples of layers: data sampling loops, metadata and token selection configurations workflow:

1. unsupervised agents embedding layer for token range and clusters.

- **6 agent** encoders (even number) for data sampling. (Token data - Lead agent/s observe)

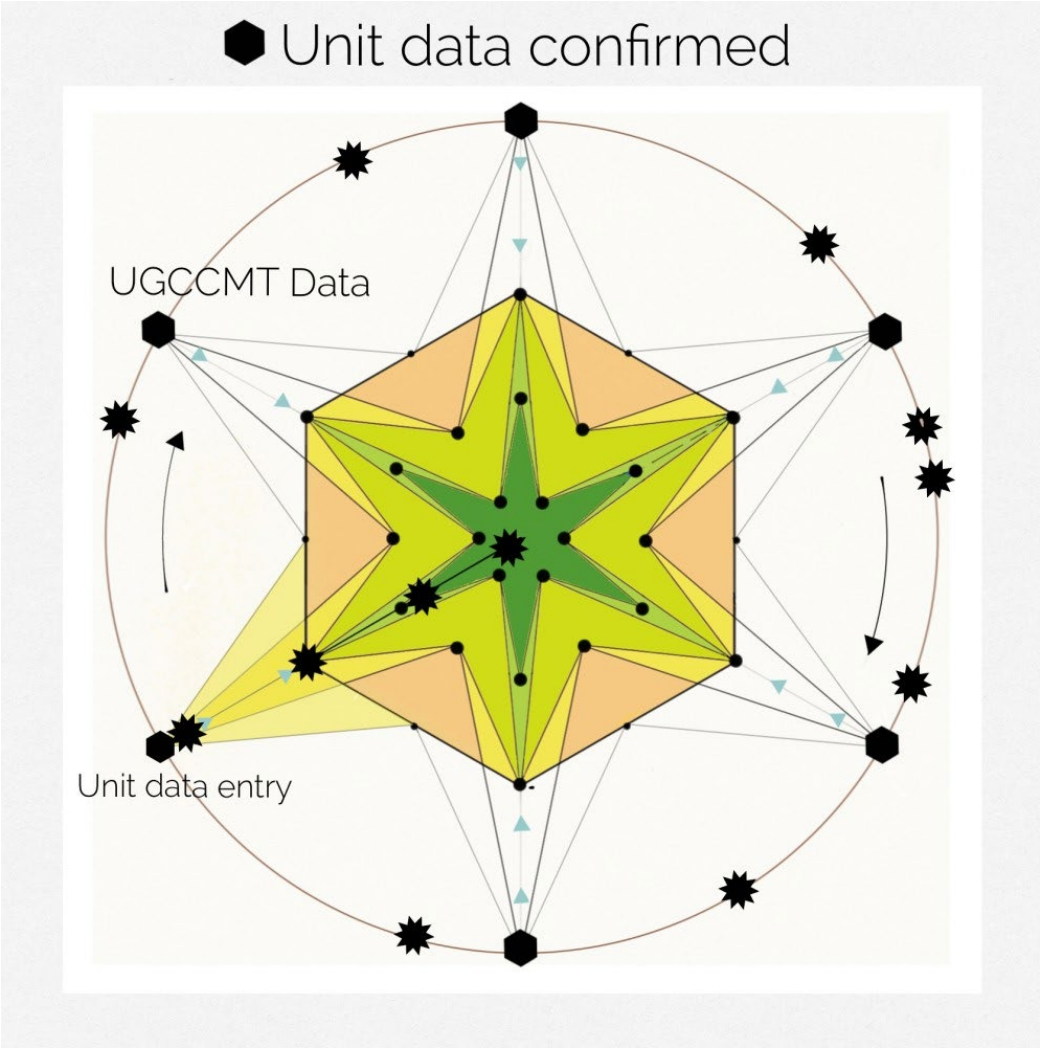
2. supervised agents embedding layer for token range and clusters.

- **6 agent** decoder - **5 agents** (odd number) and operations token data - **lead agent**/s labelling.

3. supervised encoder/decoder layer for operating system database updates.

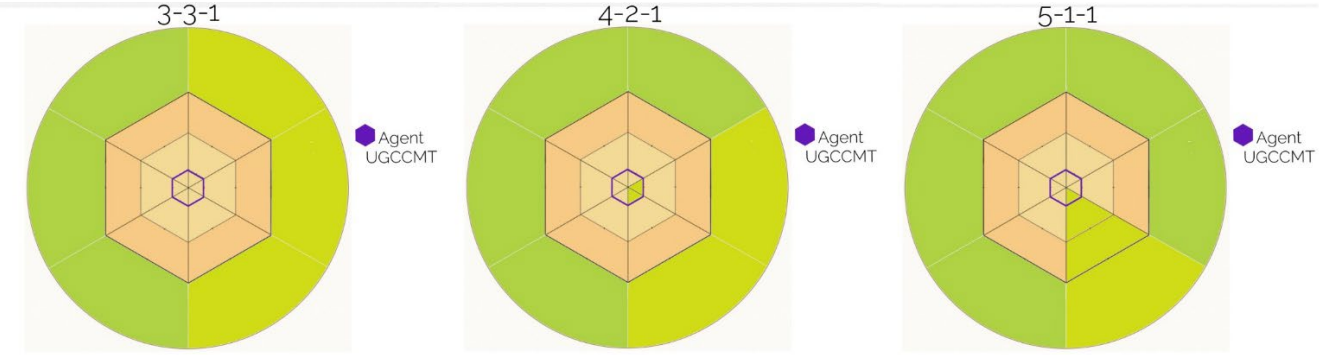
- The Operations **lead-agent** and 5 **assistants** (odd number) token memory variables. For example, 5, 10, or 50k token vocabulary.

1. Unsupervised agents embedding layer for (2D flat data) token range and clusters.



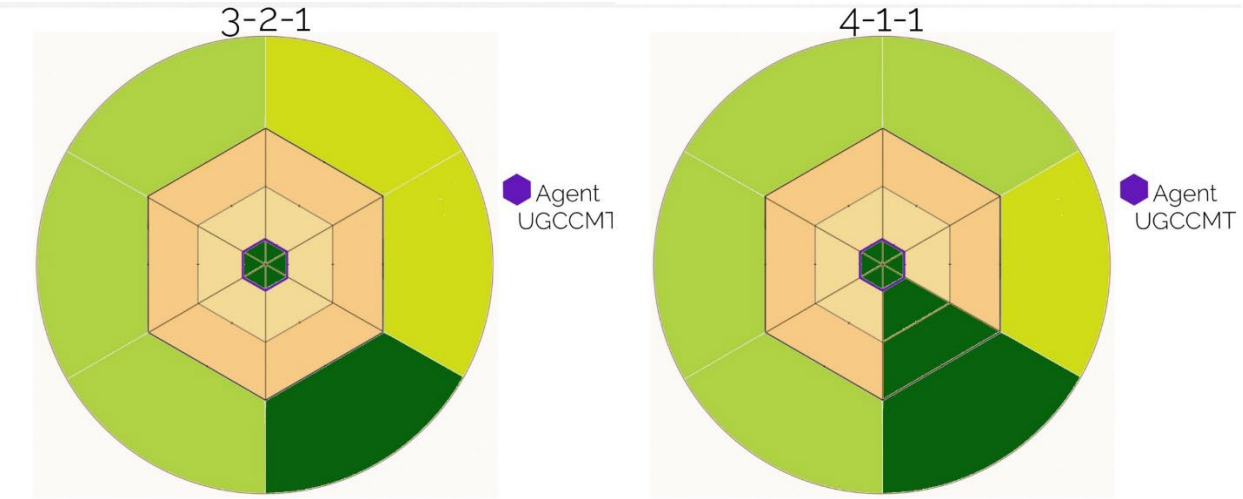
Images created by the author

6 agents only (even number) for label evaluation and classification (e.g., data types, metadata and clear label of token supervision colour code categories) in each unit **Group.**



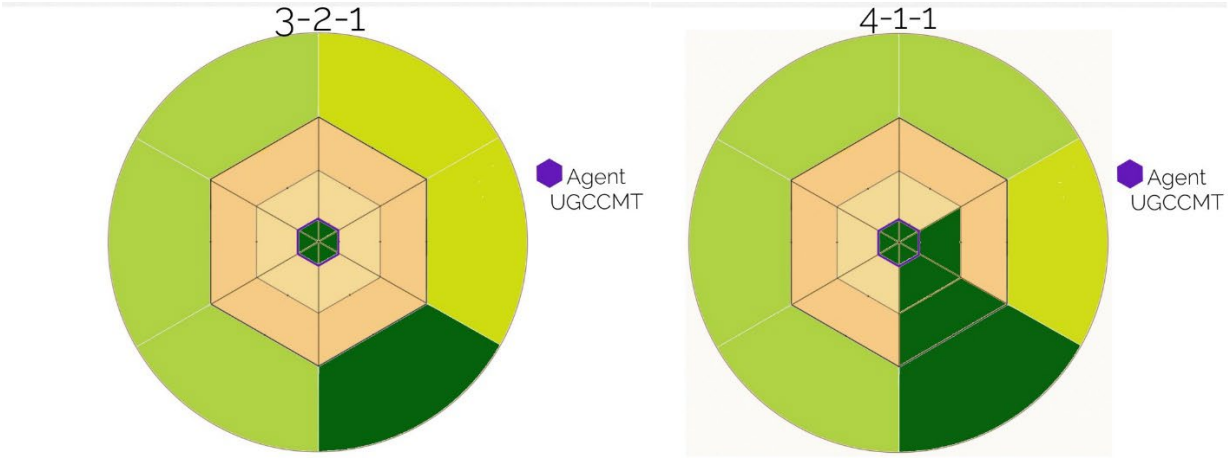
- 1st Substack layer 1 - 3-3-1 pre-structured data assessment
- 2nd Substack layer 1 - 4-2-1 tagging structured data
- 3rd Substack layer 1 - 5-1-1 pre-defined structured data classification for processing in round 2 Supervised agents

2. Supervised agents embedding layer for token range and clusters.
5 agents (odd number) and lead agent decision-making (e.g., data types, metadata, clear labelling and data categories) in each **UNITs** group.



Images created by the author

3. Supervised encoder/decoder layer for the operating system database and multimodal agent updates.
Lead agent, five assistants (odd number), decision-making, and supervision colour code cluster and sub-cluster tokens.

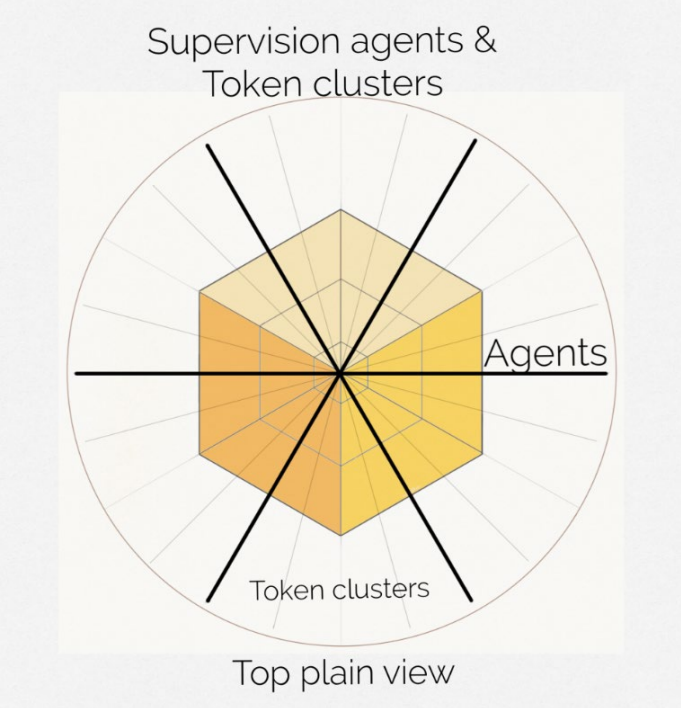


Images created by the author

- 1st Substack layer 3 - 3-2-1 encoding data locations UGCCMT.
- 2nd Substack layer 3 - 4-1-1 encoding token **data finetuning** (dark green) sub-cluster and sub-token (light green) **metadata** spiked token to sphere surface for further fine-tuning.

All data tokens (unconfirmed/curated) past the regular hexagon are spiked data tokens (class labelling UGCCMT) for the pretraining consensus process 3.1

5.1 3D fixed sphere (State 1) Top plain view and system data updates.



State 1, Based on quantum computing theory, Superposition 0 or 1 location - The top-down view of a fixed sphere shows 18 token clusters that connect with six agents, which triggers spiked data flow to the outer sphere for machine learning, processing **system updates, and model fine-tuning.**